

Mastering Java

A Compressed Book for Experienced Programmers

Patrick Lin

July 22, 2026

Contents

Introduction: What the Compiler Proves and the Running Program Does	1
Pedagogical diagnosis and design decision	1
1 Variables, Values, References, and Object Identity	3
Compressed thesis	3
1. Typed storage locations hold values	3
2. Reference values preserve access to objects	4
3. The assignment target determines what changes	5
4. Parameters receive argument values	6
5. <code>final</code> restricts a location, not a reachable graph	7
6. Identity and logical equality ask different questions	8
7. Array copies can preserve nested aliases	9
False models, repaired	10
Mastery questions	11
Implementation lab — Copy-on-Write Profile Update	11
Presentation prompt	11
Ready-when test	12
2 Expressions, Conversions, and Numeric Results	13
Compressed thesis	13
1. Context determines which conversions are available	13
2. Operation width is chosen before assignment	14
3. Floating-point values approximate real arithmetic	15
4. Narrowing is a representation rule, not validation	16
5. Shift operators use the left operand’s promoted width	16
6. Grouping and evaluation order answer different questions	17
7. Abrupt completion stops the remaining expression	18
False models, repaired	19
Mastery questions	19
Implementation lab — Overflow-Safe Basis-Point Calculation	20
Presentation prompt	20
Ready-when test	20
3 Control Flow, Scope, and Definite Assignment	21
Compressed thesis	21
1. Blocks define lexical regions for local names	21
2. Boolean operators remove or preserve execution paths	22
3. Local reads require definite assignment	23
4. Abrupt completion changes later reachability	24
5. Pattern-variable scope follows successful matches	24
6. Loop exits determine what is known afterward	25
7. Switch form determines fall-through and result paths	26
False models, repaired	27

Mastery questions	28
Implementation lab — Flow-Safe Status Rendering	28
Presentation prompt	28
Ready-when test	29
Personal appendix index	30
Glossary	31

Introduction: What the Compiler Proves and the Running Program Does

Java source participates in several distinct mechanisms. The language compiler checks declarations, expression types, accessibility, definite assignment, and other source rules. The virtual machine loads, links, initializes, and executes classes. Objects preserve identity while reference values move through variables, fields, arrays, and parameters. Library contracts add meanings—such as collection equality, resource ownership, or executor shutdown—that the language alone does not invent. Concurrent threads add ordering and visibility constraints that a locally correct method cannot settle by itself. Reliable Java reasoning begins by naming which mechanism performs an operation, on which value or object, during which phase, and what failure remains possible elsewhere.

This book is compressed by design. Each chapter states a durable account, then makes the account retrievable through committed predictions, exact evidence, causal explanations, an independent lab, delayed questions, and a short oral presentation. Do not read a prediction case as ordinary exposition. Stop before its result, write the result you expect, and name the rule responsible. A wrong prediction is useful evidence only if it remains visible long enough to locate the first mistaken step.

The route has five ordered regions. **Values and execution (Chapters 1–4)** establishes storage, identity, expressions, control flow, and calls. **Object and type models (Chapters 5–8)** connects construction, inheritance, domain forms, and erased generics. **Data, effects, and failure (Chapters 9–12)** applies those mechanisms to equality, collections, resources, exceptions, lambdas, and streams. **From source to concurrent run time (Chapters 13–16)** follows code through class loading, modules, the memory model, and task execution. **Production correctness (Chapters 17–20)** tests the combined model at external boundaries and in a concurrent capstone. The detailed contents are not a list of independent features; each region changes what the conclusions of the preceding regions mean in a larger program.

Pedagogical diagnosis and design decision

The intended reader can often make Java code compile but cannot always say why a nearby variation fails, which overload is selected, whether two paths reach the same object, when a generic guarantee disappears, or what another thread may observe. Syntax-first tutorials help a new programmer produce the first program. Encyclopedic references help a reader find a named feature. Recipe collections help reproduce a familiar solution. None directly repairs a model assembled from slogans whose hidden boundaries surface only when features interact.

This sequence begins with variables, values, references, and object identity because every later region depends on the difference between a copied value and a shared object. It then moves through the operations that transform those values before introducing larger abstractions. Generics arrive only after declared and run-time types can be separated. Streams arrive only after mutation and collection contracts are visible. Concurrency arrives only after object state, initialization, failures, and execution phases have concrete meanings. The sequence makes each later guarantee state exactly what it adds and what it still cannot add.

The book is trying to convey a method of technical judgment as much as a body of Java facts: predict the event, identify the responsible rule or component, inspect decisive evidence, locate the first divergence, and defend the conclusion's boundary. That method prevents a compiler success from being mistaken for input validation, a

test from being mistaken for a universal proof, an immutable view from being mistaken for unique ownership, or synchronization in one method from being mistaken for a whole-operation invariant.

Dense canonical prose serves readers who already know how ordinary code looks. It should not force every reader through the same remedial detour. When a particular passage remains opaque after an honest prediction and reconstruction attempt, the reader can record the passage, expected result, observed result, and remaining confusion. A personal appendix may then decompress exactly that missing dependency. Repeated reports are evidence for revising the shared chapter; one local difficulty is not a reason to turn the entire book into a beginner sequence.

The final standard is independent use. A reader who merely recognizes each term has not yet mastered the material. A ready reader can analyze an unfamiliar case without the page in view, produce an implementation that preserves its contracts, diagnose the first wrong assumption when evidence disagrees, and say precisely what the result does not establish.

Chapter 1

Variables, Values, References, and Object Identity

Chapter type: Acquisition

Prerequisites: Ordinary declarations, assignments, method calls, simple classes, and arrays

Capabilities introduced: Predict value copying, object aliasing, mutation, reassignment, parameter effects, `final` restrictions, reference identity, logical equality, and shallow array copying

Earlier chapters integrated: None

Later chapters enabled: 2–5, 8–10, 15, and every chapter that reasons about state

Estimated study time: 4–6 hours across four sessions

Runnable sources: `examples/ch01/src/`, `examples/ch01/test/`, and `examples/ch01/lab/`

Compressed thesis

A Java variable is a typed storage location. Its current value is either a primitive value or a reference value. A reference value may refer to an object—a class instance or an array—or it may be the null reference, which refers to no object. Assignment copies a value into the variable, field, or array component selected by the left-hand side. Copying a reference value does not copy the object; it can create another access path to the same object. A state change made through one such alias is observable through the others.

A method invocation also passes values. Each parameter is a new variable for that invocation, initialized with the corresponding argument value. Reassigning a reference parameter therefore changes only the parameter variable, while writing a field or array component through the parameter can mutate the caller's object. A `final` reference prevents later assignment to that particular variable; it does not make the referenced object immutable or remove other aliases.

`Reference ==` asks whether two values are the same reference or are both null. An `equals` method asks whatever logical-equivalence question its class contract defines. Array cloning creates a new outer array and copies its component values, so a cloned array of references can still share all of its element objects. These rules describe Java-language behavior and an operational object graph; they do not claim a particular physical heap layout or an ownership system that Java does not provide.

1. Typed storage locations hold values

The Java Language Specification defines a variable as a storage location with an associated type. Local variables, parameters, instance fields, static fields, and array components are all variables, but they are created and initialized under different rules. A local variable must be definitely assigned before use. Fields and array components receive default values when their containing object or array is initialized. Later chapters will trace those timing rules; this chapter begins with the common fact that an assignment stores a value in a selected location.

Java divides values into two families. A variable of primitive type holds a value of that exact primitive type. The primitive types are `boolean`, `byte`, `short`, `char`, `int`, `long`, `float`, and `double`. A variable of reference type holds either a reference to a compatible object or the null reference. Class types, interface types, type variables, and array types are reference types.

Do not collapse a variable, its value, and a referenced object into one entity. In the declaration `int count = 7`, `count` is a variable and 7 is the primitive value stored there. In `Counter counter = new Counter(1)`, `counter` is a variable, the value stored in it is a reference, and the `Counter` created by `new` is an object with its own identity and field state.

Predict before running: After `second` is incremented, what values do the two variables hold? Which location did `++` update?

```
int first = 7;
int second = first;
second++;

System.out.printf("primitive: first=%d second=%d%n", first, second);
```

Output:

```
primitive: first=7 second=8
```

The initializer for `second` copies the current `int` value from `first`. The postfix increment writes a new `int` value into `second`; it does not mutate the number seven and cannot redirect the already completed initialization of `first`. Primitive assignment therefore cannot create shared mutable object state. Two primitive variables may contain equal values, but neither is an access path to the other.

The declared type matters before execution. It limits which values a variable may hold and which operations an expression may request. Chapter 2 will derive conversions in detail. For now, keep the phase boundary visible: a successful assignment means the compiler accepted the source relationship, subject to special boundaries such as unchecked operations and run-time array checks. It does not mean an incoming value satisfies an application rule such as “port is between 1 and 65535.”

2. Reference values preserve access to objects

An object is either a class instance or an array. Each successful object-creation operation produces an object with identity. Object state consists of its instance fields or, for an array, its component variables. A reference value permits operations on the object to which it refers. The language permits many reference values to refer to the same object.

Two access paths are **aliases** when their current reference values refer to the same object. Aliasing is a relationship among paths and one object, not a claim that the variables have merged. Each variable remains a separate storage location and can later receive a different reference value if assignment is permitted.

Predict before running: Does incrementing through `alias` change the value later read through `primary`? Are `primary` and `alias` the same variable?

```

Counter primary = new Counter(1);
Counter alias = primary;
alias.value++;

System.out.printf(
    "alias: same=%b primary.value=%d alias.value=%d\n",
    primary == alias,
    primary.value,
    alias.value
);

```

Output:

```
alias: same=true primary.value=2 alias.value=2
```

The assignment to `alias` copied the reference value held by `primary`. Only one `Counter` expression executed, so only one `Counter` object was created. The field update selected the `value` field inside that object. A later read through either reference observes the field's new value. `primary == alias` is true because both operands produce references to the same object, not because the two local variables are one location.

An object graph is a useful operational representation. Draw variables and object fields as labeled locations. Draw an object as an identity-bearing node. A primitive value may be written directly in a location; a non-null reference value can be drawn as an edge to a node. In the preceding case, `primary` and `alias` have separate edges to the same `Counter` node, whose `value` field contains 2. The graph predicts assignment, identity, and mutation. It does not assert whether a virtual machine uses a raw address, a handle, compressed object pointers, escape analysis, or another physical representation.

The null reference is a reference value that reaches no object. It can be copied and compared. An operation that needs an object, such as instance field access or instance method invocation, cannot follow null to a node and generally completes abruptly with `NullPointerException`. Null is not an object with an empty state, and two null-valued variables do not alias an object merely because `a == b` is true. Chapter 11 handles the failure boundary; Chapter 19 handles API null policy.

3. The assignment target determines what changes

The slogan “assignment changes the variable” is too broad because fields and array components are also variables. The left-hand operand identifies the location to update. If `primary` is a local variable, `primary = replacement` stores a different reference value in that local variable. If `primary.value` is an instance field, `primary.value = 9` follows `primary` to an object and stores an `int` in that object's field. If `items[0]` is an array component, `items[0] = replacement` changes an element location inside the array object.

This book uses **reassignment** for storing a new value in an existing location and **rebind** informally when a reference variable changes which object it reaches. Mutation means a state change to an existing object. Rebinding a local reference changes one edge in the object graph. Mutating a field changes state inside a node. Assigning a reference into an object field is a mutation of the containing object and also redirects one edge from that object. Naming the exact location avoids treating every `=` as the same state transition.

Predict before running: After the second statement, which object does `left` reach? Which object does `right` reach? What happened to the original object's field?


```
Counter left = new Counter(3);
Counter right = left;

left = new Counter(40);
right.value = 4;

System.out.printf(
    "reassign: same=%b left.value=%d right.value=%d\n",
    left == right,
    left.value,
    right.value
);
```

Output:

```
reassign: same=false left.value=40 right.value=4
```

The second `new` expression creates a second object, and assignment redirects only `left`. `right` continues to refer to the first object. The field assignment through `right` changes that first object's state. Nothing in the reassignment to `left` deletes or resets the first object; whether an object later becomes eligible for garbage collection depends on reachability through all live paths, not on one variable name. Collector algorithms and liveness diagnostics are outside this chapter.

Aliasing is not itself a defect. Two collaborators may intentionally share a connection, cache entry, mutable buffer, or domain entity whose stable identity matters. Aliasing enlarges the reasoning boundary: to prove that shared state stays unchanged, a programmer must account for every path that can perform a relevant mutation during the relevant lifetime. An API therefore needs a mutation and ownership policy even though Java's ordinary reference types do not encode unique ownership.

4. Parameters receive argument values

For each method invocation, Java creates a new parameter variable for every declared parameter. Each parameter is initialized with the corresponding argument value. That rule applies uniformly: an `int` argument supplies an `int` value; an object-producing argument supplies a reference value; a null-valued argument supplies null. Java does not give the method an assignable alias for the caller's local variable.

Predict before running: The method mutates through its parameter and then reassigns the parameter. What state remains in `caller`? What state is in the returned object? Are they identical?

```

static Counter mutateThenReplace(Counter parameter) {
    parameter.value += 10;
    parameter = new Counter(99);
    parameter.value++;
    return parameter;
}

Counter caller = new Counter(2);
Counter returned = mutateThenReplace(caller);

System.out.printf(
    "call: caller=%d returned=%d same=%b%n",
    caller.value,
    returned.value,
    caller == returned
);

```

Output:

```
call: caller=12 returned=100 same=false
```

Argument evaluation copies the reference to the first `Counter` into the new parameter variable. The first field assignment follows that reference and mutates the shared object from 2 to 12. The next assignment stores a reference to a newly created object in the parameter variable only. The caller’s local variable is a different location, so it still reaches the first object. The final increment changes the second object, and the return statement copies its reference value into the calling expression; initialization then stores that value in `returned`.

“Objects are passed by reference” predicts the first mutation but commonly predicts that `parameter = new Counter(99)` should redirect `caller`. That prediction is false. “Object references are passed by value” predicts both operations, provided the sentence is unpacked: the passed value is a reference, copying it creates an alias, field access can mutate the shared object, and assignment to the parameter changes only the call-local parameter variable.

A method can request that a caller adopt a replacement object by returning its reference, but the caller performs the assignment: `caller = mutateThenReplace(caller)`. The method cannot directly reassign that caller local through an ordinary Java parameter. A method can still modify caller-visible locations reached through references—for example, an element of a supplied array or a field of a supplied holder object. That is mutation of a shared object, not pass-by-reference parameter semantics.

5. `final` restricts a location, not a reachable graph

A `final` variable may be assigned only under the language’s final-variable rules. Once a final local variable has been assigned, it cannot be the target of a later assignment or increment. For a reference variable, this protects one edge in the object graph: the variable cannot later store null or a reference to a different object. It does not freeze the object’s fields, recursively freeze referenced objects, prevent methods from mutating the object, or prevent another alias from doing so.

Predict before running: Does this compile? If it does, what value is printed? Which statement would fail compilation if uncommented?

```
final Counter stableReference = new Counter(5);
Counter writableAlias = stableReference;

writableAlias.value += 2;
stableReference.value++;
// stableReference = new Counter(0);

System.out.printf("final: value=%d%n", stableReference.value);
```

Output:

```
final: value=8
```

Both field assignments are legal because neither assigns the local variable `stableReference`; they update the field of the shared `Counter`. The commented assignment is illegal because it attempts to store a new reference value in a final local variable. The compiler enforces that restriction at the assignment target. The object has no general knowledge that one path reaching it was stored in a final variable.

Java uses `final` for several distinct declarations: local variables, parameters, fields, methods, and classes. Their restrictions are related by the keyword but not interchangeable. A final method cannot be overridden, and a final class cannot be subclassed; neither claim makes every instance deeply immutable. A final field also participates in special construction and memory-model rules, but Chapter 5 and Chapter 15 must establish the conditions before this book relies on those guarantees.

An immutable design requires a contract about observable state, not merely a final reference. Such a design may combine final fields, construction invariants, defensive copies, immutable component types, no mutating methods, and safe publication. Its depth is explicit: an immutable outer object can still expose a mutable array and thereby allow observable change. Chapter 19 turns those observations into an API policy.

6. Identity and logical equality ask different questions

For reference operands, `==` returns true when both values are null or both refer to the same object. It does not inspect fields. Logical equivalence is expressed by an `equals` method whose class-specific contract decides which state matters. The default `Object.equals` behavior is identity equality, but classes may override it. `String`, for example, defines equality in terms of its character sequence.

Predict before running: Which comparisons are true? Why can the two `String` values be logically equal without being identical?

```
String textA = "Java";
String textB = new String("Java");

Counter counterA = new Counter(1);
Counter counterB = new Counter(1);
Counter counterAlias = counterA;

System.out.printf(
    "equality: textIdentity=%b textEquals=%b counterIdentity=%b aliasIdentity=%b%n",
    textA == textB,
    textA.equals(textB),
    counterA == counterB,
    counterA == counterAlias
);
```

Output:

```
equality: textIdentity=false textEquals=true counterIdentity=false aliasIdentity=true
```

The string constructor creates a distinct `String` object, so the reference identity comparison is false. Both strings represent the same character sequence under `String.equals`, so logical equality is true. The two `Counter` constructions also produce distinct objects. `Counter` does not override `Object.equals` in the runnable example, so even `counterA.equals(counterB)` would remain false despite equal-looking fields. The alias comparison is true because it compares two references to the one object created for `counterA`.

String literals introduce interning rules, but code should not use `==` as content comparison merely because two literal references may be identical. That observation depends on how the strings were obtained and answers the identity question. The desired application question is normally sequence equality, which `String.equals` defines directly.

Full equality design requires more structure than this chapter has introduced. An overridden `equals` method must obey its contract, equal objects must produce equal hash codes, and ordering may need to agree with equality. Mutating state used by equality can break collection lookups. Chapter 9 acquires those relationships. The durable rule here is smaller: decide which question the program needs, then use the operation whose contract asks that question.

7. Array copies can preserve nested aliases

An array is an object, and every array component is a variable. An array of primitive type stores primitive values in its components. An array of reference type stores reference values. A two-dimensional array type such as `int[][]` is therefore an array whose components contain references to `int[]` objects; Java does not require one rectangular block containing every integer.

Calling `clone()` on an array creates a new array object of the same run-time type and copies the component values. For a primitive array, those copied components are primitive values. For an array of references, the copied components are reference values, so the old and new outer arrays initially refer to the same element objects. This is a shallow copy.

Predict before running: Which arrays have new identity? Which row remains shared? Which assignment affects the original grid, and which affects only the copied outer array?

```

int[] [] grid = {
    {1, 2},
    {3, 4}
};

int[] [] outerCopy = grid.clone();
outerCopy[0][0] = 9;
outerCopy[1] = new int[] {7, 8};

System.out.printf(
    "arrays: outerSame=%b firstRowSame=%b original00=%d original10=%d copy10=%d%n",
    grid == outerCopy,
    grid[0] == outerCopy[0],
    grid[0][0],
    grid[1][0],
    outerCopy[1][0]
);

```

Output:

```
arrays: outerSame=false firstRowSame=true original00=9 original10=3 copy10=7
```

The clone operation creates a new outer array, so `grid == outerCopy` is false. Copying component zero copies a reference to the first row, so both outer arrays initially reach the same `int[]`; changing its component zero is visible through `grid`. Assigning `outerCopy[1]` changes a component of the copied outer array, redirecting only that component to a new row. The original outer array still refers to its original second row.

The phrase “copy the array” is incomplete until it names which array objects must be new. A fully independent copy of this particular graph requires a new outer array and a new row array for every row that must not be shared. General object graphs can contain cycles, repeated aliases, objects with identity-sensitive contracts, and resources that cannot sensibly be duplicated. Java supplies no universal deep-copy operator because “deep” is a domain policy, not merely a recursion depth.

Selective copying can be useful. If a transformation changes only one row and the program guarantees that unchanged rows will not be mutated, a new outer array plus one new row may preserve safe sharing and avoid unnecessary work. If callers can mutate the shared rows, the same topology violates snapshot independence. The operation is identical; the surrounding mutation contract determines whether the sharing is valid.

False models, repaired

- **“A reference variable contains the object.”** It contains a reference value; the object has a separate identity and may be reached through other values.
- **“Assigning an object variable copies the object.”** Assignment copies the current reference value and may create another alias.
- **“Java passes objects by reference.”** Invocation initializes a new parameter variable with the argument value; a reference value can preserve access to a shared object, but parameter reassignment stays local.
- **“Every assignment is a rebind.”** The left-hand side may select a local, a field, or an array component. A field or component assignment mutates its containing object.
- **“final makes an object immutable.”** A final reference variable cannot later hold another value; reachable object state may still change.
- **“== compares object contents.”** Reference `==` compares identity or two null references. Logical equality comes from the class’s `equals` contract.
- **“Cloning an array recursively clones its elements.”** Array cloning creates one new array and copies component values; copied references retain element aliases.
- **“Null is an empty object.”** The null reference refers to no object, so an object-requiring operation cannot follow it to state or behavior.

Mastery questions

Commit to a result and causal explanation before compiling any constructed case.

1. `long b = a; b++;` follows `long a = 4;`. Predict both values and identify every updated location.
2. Two local variables refer to one mutable `Box`. One variable receives a new `Box`; the other changes a field. Draw the graph after each statement.
3. A method sets `parameter.enabled = true`, assigns `parameter = null`, and returns. What can the caller observe, and why does no caller variable become null?
4. Give one concrete observation that the phrase “objects are passed by reference” predicts incorrectly.
5. A final local refers to an array. Which of these are permitted: replacing the local’s value, replacing element zero, and changing a field of the object stored at element zero? Name the assignment target in each case.
6. Two separately constructed `Customer` objects contain the same database ID. What does `==` establish? What additional contract would be required for logical equality?
7. Why does `new String("x")` make an identity comparison a different question from `String.equals`?
8. Draw a graph for `Widget[][] copy = original.clone()`. Which nodes are necessarily new, and which may be shared?
9. Diagnose a function advertised as returning an independent snapshot when it clones only an outer array of mutable objects.
10. Design a selective copy for a three-level configuration graph where only one leaf changes. Which path objects must be new if old and new roots must be independently stable?
11. A variable has interface type `Runnable` and refers to an instance of a named class. Separate the variable’s compile-time type, its current reference value, the object’s identity, and its run-time class.
12. Explain why a final reference can reduce accidental reassignment without reducing the number of aliases that can mutate the object.
13. A null-valued variable compares equal with another null-valued variable. Why is “the two variables alias the same null object” an invalid explanation?
14. Describe one API where shared mutation is intentional. State which aliases may mutate, during what lifetime, and which invariant contains the risk.

The closed-book workspace, transfer cases, mismatch-report format, and retrieval schedule are in `exercises/ch01.md`.

Implementation lab — Copy-on-Write Profile Update

Implement `StudentProfileUpdater.updateTheme` in `examples/ch01/lab/`. The input graph contains a `Profile`, its mutable `Preferences`, and an audit-code array. Return a new outer `Profile` with the requested theme without mutating the input. The result must contain a new `Preferences` object, because its state differs. Preserve the `String` ID reference and the audit-array reference, because the contract designates both as unchanged shared values. This is selective copying, not a claim of deep immutability: callers capable of mutating the shared audit array can still affect both profiles.

Before coding, commit to all four identity relationships: input/result profiles, input/result preferences, input/result IDs, and input/result audit arrays. The evaluator also checks values and proves the input preferences were not mutated. After your implementation passes, run the evaluator against `FaultyProfileUpdater`, which creates a new outer profile but mutates the still-shared preferences object. Explain the first operation at which that implementation violates the contract.

Presentation prompt

Give a 10–15 minute explanation from sparse notes. Reconstruct the relationship among a variable, its type, its current value, a reference, and an object. Include one committed prediction involving method-parameter reassignment, a graph derivation for the two-dimensional array clone, the false model that `final` means immutable, the boundary between `==` and `equals`, one legitimate use of shared mutation, and one fact the Chapter 1 tests cannot prove universally. End by naming the mechanism that still feels least reliable. The recording checklist is in `presentations/prompts/ch01.md`.

Ready-when test

Proceed when, without the chapter open, you can predict every canonical output; draw the object graph after reassignment, mutation, a method call, and an array clone; explain pass-by-value without a special exception for objects; separate a final variable from object immutability; choose deliberately between identity and logical equality; implement and pass the copy-on-write lab; diagnose the faulty updater at its first divergent write; and state what the compiler and tests establish and what they do not. A vocabulary-perfect answer that predicts the wrong object identity or mutation is not ready.

Chapter 2

Expressions, Conversions, and Numeric Results

Chapter type: Acquisition

Prerequisites: Chapter 1

Capabilities introduced: Derive expression types and values, numeric promotion, overflow, narrowing, floating-point results, shifts, string concatenation, boxing, unboxing, and side-effect order

Earlier chapters integrated: Variables, copied values, references, and identity

Later chapters enabled: 3–4, 8, 12, and 18

Estimated study time: 4–6 hours across four sessions

Runnable sources: `examples/ch02/src/`, `examples/ch02/test/`, and `examples/ch02/lab/`

Compressed thesis

Every non-void Java expression has a compile-time type, and evaluating it either produces a value of that type or completes abruptly by throwing an exception. Derive a numeric expression in stages: determine the grouping, evaluate operands in Java’s specified order, apply the conversions required by the expression’s context, perform the operation in the promoted type, and only then convert the result for an assignment, cast, invocation, or string context. The destination variable does not retroactively enlarge an operation that has already overflowed.

Java’s primitive numeric types are finite representations, not mathematical number sets. Integer operations use `int` or `long` precision and silently discard high-order bits on overflow. Floating-point operations use finite IEEE 754 values and rounding, so many decimal fractions are not exact. A cast changes representation according to a specified conversion; it does not prove that the source value satisfies an application domain. Boxing produces a wrapper reference whose numeric value is defined but whose identity is generally not a valid program assumption. These boundaries turn expression prediction into a mechanical trace rather than an intuition about arithmetic.

1. Context determines which conversions are available

Chapter 1 separated a variable from the value stored in it. An expression adds a third entity: a computation with its own compile-time type. The type of the destination and the type of the expression are not necessarily the same. An assignment context may convert the completed expression value before storing it, but the operator first follows the rules for its operands.

Numeric arithmetic promotes `byte`, `short`, and `char` operands to `int` unless a `long`, `float`, or `double` operand selects a wider promoted type. Consequently, adding two `byte` values produces an `int`, even when the particular result would fit in a `byte`. Assignment of certain `int` constant expressions is a separate rule: the compiler may narrow a representable constant to `byte`, `short`, or `char`. It does not extend that permission to an ordinary run-time expression.

Predict before running: What are the types of `promoted` and the addition inside the cast? Why does the same numeric sum appear as two different stored values?

```
byte left = 100;
byte right = 30;

int promoted = left + right;
byte narrowed = (byte) (left + right);
byte constant = 100 + 20;

System.out.printf(
    "conversion: promoted=%d narrowed=%d constant=%d%n",
    promoted,
    narrowed,
    constant
);
```

Output:

```
conversion: promoted=130 narrowed=-126 constant=120
```

Both additions involving variables undergo binary numeric promotion and execute as `int` addition. The assignment to `promoted` stores 130 unchanged. The explicit cast narrows that completed `int` value to eight bits; the resulting bit pattern denotes -126 as a `byte`. The initializer for `constant` is a compile-time constant expression whose value is representable as `byte`, so assignment conversion permits it without a cast.

The corresponding method `byte add(byte a, byte b) { return a + b; }` does not compile: the return context cannot silently narrow the nonconstant `int` result. The negative compilation fixture in `examples/ch02/test/` verifies that diagnostic. A cast can make the method compile, but that changes the program from “reject an out-of-range result” to “retain only the low eight bits.” Those are different contracts.

Compound assignment contains an implicit conversion. If `byte total = 120;`, then `total += 10` behaves like `total = (byte) (total + 10)`, except that the left-hand side is evaluated once. It therefore compiles and stores -126. The shorter syntax is not evidence that the operation stayed within the range of `byte`.

2. Operation width is chosen before assignment

For integer operators other than shifts, an operation executes as `long` when at least one promoted operand is `long`; otherwise it executes as `int`. The type on the left of an assignment receives only the result that the operator has already produced. A later cast of that result likewise cannot recover bits discarded by an earlier `int` operation.

Predict before running: Which multiplication is performed as `int`, and which is performed as `long`? Does the `long` destination protect the first expression?

```
int factor = 50_000;

long lateWidening = factor * factor;
long earlyWidening = (long) factor * factor;

System.out.printf(
    "width: late=%d early=%d%n",
    lateWidening,
    earlyWidening
);
```

Output:

```
width: late=-1794967296 early=2500000000
```

In the first initializer, both operands have type `int`, so multiplication executes in 32-bit precision. The mathematical product exceeds `Integer.MAX_VALUE`; the integer operation retains the low 32 bits, which represent `-1794967296`. Assignment then widens that already incorrect-for-the-domain `int` value to `long`. In the second initializer, the cast converts the left operand to `long` before multiplication. Binary numeric promotion widens the other operand, and the product is computed in 64-bit precision.

Java integer overflow does not itself throw an exception or record a flag. Whether wraparound is correct depends on the domain. Hash calculations and some bit algorithms may require modular arithmetic. A financial amount, byte count, or array-size calculation usually requires a range invariant instead. Choose an operation that enforces that invariant, such as a wider proven-safe intermediate, a precondition, or an exact-arithmetic method that throws on overflow. Merely storing the result in a wider variable is too late.

Width alone is not a proof of safety. The product of any two `int` values fits in a `long`, which supports this chapter's lab. The product of arbitrary `long` values does not necessarily fit in a `long`. A sound derivation names the operand ranges as well as the primitive types.

Integer division also follows the promoted operand types. `5 / 2` produces the `int` value 2; assigning it to `double` produces `2.0`, not `2.5`. An operand must be floating-point before division, as in `5 / 2.0`, if the operation is intended to use floating-point division.

3. Floating-point values approximate real arithmetic

`float` and `double` have finite sets of values. Java floating-point expressions use IEEE 754 binary formats and specified rounding. A decimal literal such as `0.1` is converted to the nearest representable binary floating-point value of its type; most finite decimal fractions do not have a finite binary expansion. Arithmetic combines those represented values and rounds each result to the result type.

Predict before running: Does three additions of `0.1` produce a value equal to the literal `0.3`? What does the longer decimal rendering reveal?

```
double accumulated = 0.1 + 0.1 + 0.1;

System.out.printf(
    "floating: value=%.17f equal=%b%n",
    accumulated,
    accumulated == 0.3
);
```

Output:

```
floating: value=0.30000000000000004 equal=false
```

The comparison is deterministic, but “floating point is random” is not its cause. Each operand and operation follows defined representation and rounding rules. The accumulated result and the nearest `double` for the literal 0.3 are adjacent unequal values in this case. A tolerance comparison may be appropriate for a measured quantity, but the tolerance must come from the domain’s acceptable error; a universal epsilon cannot decide every scale or calculation.

Floating-point division by zero does not follow integer division’s exception rule. It can produce positive or negative infinity, and operations without a unique numerical result can produce NaN. NaN is unordered: comparisons such as `<`, `>`, and `==` involving NaN are false. This chapter establishes the language behavior, not a complete numerical-analysis method. Decimal financial arithmetic, error propagation, and arbitrary-precision types require their own domain contracts and are deliberately outside this chapter.

4. Narrowing is a representation rule, not validation

A numeric cast requests a conversion that an assignment context may not allow implicitly. For integer-to-integer narrowing, high-order bits can be discarded. For floating-point-to-integer narrowing, Java first rounds toward zero; values outside the target range are then clamped to the nearest endpoint, and NaN converts to zero. None of these outcomes means that the source belonged to an application-valid interval.

Predict before running: Do the casts throw? Which operations discard a fraction, clamp a magnitude, or retain low bits?

```
int truncated = (int) -3.9;
int clamped = (int) 3_000_000_000.0;
byte lowBits = (byte) 258;

System.out.printf(
    "casts: truncated=%d clamped=%d lowBits=%d\n",
    truncated,
    clamped,
    lowBits
);
```

Output:

```
casts: truncated=-3 clamped=2147483647 lowBits=2
```

The first cast rounds toward zero, not toward negative infinity. The second produces `Integer.MAX_VALUE` rather than throwing. The third retains the low eight bits, producing 2. A parser that casts a `long` to `int` has not validated an `int`-sized identifier; it has defined a conversion for every source value. Validate the range before casting when loss would violate the contract.

Reference casts answer a different question and can throw `ClassCastException` after a run-time type check. Chapter 8 will acquire that relationship with subtype polymorphism. The shared lesson is narrower: a cast is an operation with precisely specified conversion or checking behavior, not a general assertion mechanism and not documentation that makes a value safe.

5. Shift operators use the left operand’s promoted width

The shift operators promote each operand separately. The result type is the promoted type of the left operand; a `long` shift distance does not make an `int` left operand into a `long`. For an `int` shift, only the low five bits of the right operand determine the distance, so the effective distance is between 0 and 31. For a `long` shift, only the low six bits matter.

>> propagates the sign bit when shifting right. >>> inserts zero bits from the left. A left shift discards bits that leave the selected width and fills low bits with zero, so it can overflow in the same modular sense as other integer operations.

Predict before running: What effective distance does 35 select for an `int`? How do signed and unsigned right shifts differ for -8?

```
int wrappedDistance = 1 << 35;
int signed = -8 >> 2;
int unsigned = -8 >>> 2;

System.out.printf(
    "shifts: distance=%d signed=%d unsigned=%d\n",
    wrappedDistance,
    signed,
    unsigned
);
```

Output:

```
shifts: distance=8 signed=-2 unsigned=1073741822
```

The distance 35 has low five bits equal to 3, so `1 << 35` has the same result as `1 << 3`. The signed shift preserves a run of leading one bits and therefore the negative sign. The unsigned shift fills with zeros, producing a large positive `int`. When a protocol assigns meaning to particular bits, state the width, signedness, mask, and valid shift-distance policy explicitly; surface syntax alone does not provide that domain description.

6. Grouping and evaluation order answer different questions

Precedence and associativity determine how tokens group into expression operands. They do not give Java permission to evaluate a right operand before a left operand. Java evaluates operator operands left to right, while respecting the grouping imposed by parentheses and precedence. Except for short-circuit and conditional operators, operands complete before the operation itself is performed.

The binary `+` operator is left-associative. If either operand of a particular `+` operation is a `String`, string conversion applies to the other operand and that operation concatenates. Once an earlier concatenation has produced a `String`, a later `+` in the same left-associated chain also concatenates.

Predict before running: Which additions are numeric? Which are concatenations? For the side-effect expression, which old values do the two postfix increments produce?

```
String first = 1 + 2 + " items";
String second = "items " + 1 + 2;

int n = 1;
int result = n++ + n++ * 10;

System.out.printf(
    "order: first=%s second=%s result=%d n=%d%n",
    first,
    second,
    result,
    n
);
```

Output:

```
order: first=3 items second=items 12 result=21 n=3
```

The first expression groups as $(1 + 2) + \text{" items"}$, so numeric addition produces 3 before concatenation. The second groups as $(\text{"items " + 1}) + 2$, so both operations concatenate. In the numeric side-effect expression, multiplication groups more tightly, but the left operand of the outer addition is evaluated first. Its `n++` produces 1 and stores 2; the multiplication's `n++` then produces 2 and stores 3, yielding $1 + 20$.

This result is specified, yet code with several mutations inside one expression forces a maintainer to simulate both grouping and time. Prefer one visible state change per statement when the compact form does not express a domain operation more clearly. The reason is auditability, not undefined Java behavior.

7. Abrupt completion stops the remaining expression

An expression completes normally when its required steps finish without an exception. If evaluation throws, it completes abruptly, and the surrounding expression does not resume its unfinished operators. Left-to-right order therefore also determines which side effects occur before a failure.

Predict before running: If the left operand throws, is the assignment in the right operand executed?

```
int marker = 0;
try {
    int ignored = fail() + (marker = 1);
} catch (IllegalStateException expected) {
    System.out.printf(
        "abrupt: type=%s marker=%d%n",
        expected.getClass().getSimpleName(),
        marker
    );
}
```

Output:

```
abrupt: type=IllegalStateException marker=0
```

The left operand invokes `fail`, which throws before the right operand begins, so `marker` remains zero. By contrast, integer division by zero is detected after both operand expressions have completed normally; side effects in both operands can occur before `ArithmeticException`. Chapter 11 develops exception propagation and recovery. The durable rule here is that a value trace must include normal versus abrupt completion at each step.

Boxing and unboxing create another abrupt boundary. Boxing converts a primitive value to a wrapper reference whose contained primitive value is specified. Separate boxing operations are not a general identity guarantee,

although the language mandates identical results for certain constant values in the range -128 through 127 and a few related constants. Compare wrapper numeric content with `equals` or by unboxing, not by reference `==`.

Unboxing calls the corresponding primitive-value operation on a wrapper reference. If that reference is null, unboxing throws `NullPointerException`. An `Integer` in arithmetic is therefore not merely an `int` with optional notation; it is a reference value whose conversion can fail before the arithmetic operator produces a result.

False models, repaired

- **“The destination type controls the arithmetic.”** Operators first choose a promoted type from their operands. Assignment conversion happens after the operation has produced a result.
- **“Widening the result to `long` prevents overflow.”** Widen an operand before the operation; widening an already overflowed `int` preserves the overflowed value.
- **“A cast validates that a value fits.”** A numeric cast performs defined truncation, clamping, or low-bit retention. Range validation is a separate program decision.
- **“Floating-point results are unpredictable.”** The representation and rounding rules are specified. The result can differ from real or decimal arithmetic without being random.
- **“A shift by 35 moves an `int` by 35 bits.”** Only the low five bits of an `int` shift distance are used, so the effective distance is three.
- **“Precedence determines execution time.”** Precedence determines grouping. Java evaluates the resulting operands left to right.
- **“Boxed numbers can be compared by `==`.”** Reference `==` asks an identity question. Most boxing conversions do not promise an identity suitable for numeric comparison.
- **“An exception from an operator means neither operand ran.”** The operator’s evaluation rule determines when failure is detected; integer division checks zero only after both operands complete.

Mastery questions

Commit to a type, value, and causal trace before compiling any constructed case.

1. Why does `long result = 60_000 * 60_000;` fail to produce the mathematical product? Give the smallest source change that selects `long` multiplication.
2. Determine whether each compiles: `byte a = 100 + 20;`, `byte b = a + 1;`, and `a += 20;`. For each accepted statement, derive the stored value.
3. Predict `5 / 2`, `(double) (5 / 2)`, and `(double) 5 / 2`. Identify when the division type is selected.
4. A service casts an incoming `long` account ID to `int`. Construct two distinct inputs that produce the same stored `int`, then state the missing invariant.
5. Explain why `(int) 1e100` does not throw and why that behavior is not range validation.
6. Predict `"x=" + 1 + 2 * 3` and `(1 + 2) * 3 + "x"`. Separate grouping, operand conversion, and operation.
7. Trace `int i = 1; int r = i++ * 10 + i++;`. Record the value produced and state written by each postfix increment.
8. Compare `-1 >> 1` with `-1 >>> 1`. Explain the result using a 32-bit representation.
9. Why do `1 << 0`, `1 << 32`, and `1 << 64` have the same `int` result? What changes if the left operand is `1L`?
10. Design a test that distinguishes late widening from early widening without relying on unspecified implementation behavior.
11. A tolerance check accepts two measured `double` values. What domain information is required to defend the tolerance, and what does the `double` type alone fail to supply?
12. In `left() / right()`, `left()` mutates state and `right()` returns zero. Which calls complete before `ArithmeticException`? Change the case so `left()` throws and derive the new trace.
13. Explain why two separately boxed `Integer` values can be numerically equal even when reference identity is not promised. Connect the answer to Chapter 1.
14. Design an overflow-safe basis-point calculation for two arbitrary `int` inputs. Prove that the chosen intermediate type can contain every possible product.

The closed-book workspace, transfer cases, mismatch-report format, and retrieval schedule are in `exercises/ch02.md`.

Implementation lab — Overflow-Safe Basis-Point Calculation

Implement `StudentRateCalculator.apply` in `examples/ch02/lab/`. Given any `int` `amount` and any `int` `basisPoints`, return the `long` result of the mathematical product divided by 10_000, using Java integer division's truncation toward zero. The multiplication must not overflow before division. The product of any two `int` values fits in `long`, so one operand conversion before multiplication is sufficient.

Before coding, predict results for a small positive case, a negative product with a remainder, 2_000_000_000 at 10_000 basis points, and the product of the two minimum `int` values. Run the evaluator against the reference implementation, then against `FaultyRateCalculator`, which performs `int` multiplication and widens only afterward. Identify the first operation where the faulty value diverges from the mathematical contract.

Presentation prompt

Give a 10–15 minute explanation from sparse notes. Reconstruct the stages from grouping through operand evaluation, conversion, promoted operation, result, and destination conversion. Include one committed overflow prediction, one floating-point derivation, the difference between precedence and evaluation order, a narrowing cast that violates a plausible domain, the identity boundary of boxing, an abrupt-completion trace, and one fact the Chapter 2 fixtures cannot prove universally. Connect the explanation to Chapter 1's distinction among variables, values, references, and objects. The recording checklist is in `presentations/prompts/ch02.md`.

Ready-when test

Proceed when, without the chapter open, you can derive the type and value of unfamiliar numeric and concatenation expressions; place a widening conversion before the operation that needs it; explain integer overflow and floating-point rounding without treating either as random; distinguish a cast from validation; predict signed and unsigned shifts including masked distances; trace left-to-right side effects separately from precedence; identify where abrupt completion stops evaluation; avoid wrapper identity as numeric equality; implement and pass the basis-point lab; diagnose the faulty implementation at its first overflowing multiplication; and state what the compiler, specification, and local fixtures each establish.

Chapter 3

Control Flow, Scope, and Definite Assignment

Chapter type: Acquisition

Prerequisites: Chapters 1–2

Capabilities introduced: Predict statement paths, determine lexical and flow-sensitive scope, prove definite assignment, distinguish reachability from execution, and trace loop and switch exits

Earlier chapters integrated: Variables, assignment targets, expression evaluation, short-circuit effects, and abrupt completion

Later chapters enabled: 4, 7, 11, and 15

Estimated study time: 4–6 hours across four sessions

Runnable sources: `examples/ch03/src/`, `examples/ch03/test/`, and `examples/ch03/lab/`

Compressed thesis

Statements organize expression evaluation into possible execution paths. At each program point, four questions are distinct: can execution reach this statement, is a declaration in scope here, has every possible incoming path assigned the local variable, and what path is taken for the current run-time values? Java answers the first three at compile time from the structure of the program. A particular successful run does not repair a missing compile-time proof.

Scope is a region of program text in which a declaration can be referred to by a simple name; it is not the same as object lifetime, storage lifetime, or run-time reachability. Definite assignment is a conservative flow analysis requiring an assignment on every possible path to a local-variable read. Reachability asks whether the language’s structural rules admit a path to a statement, not whether current inputs will take it. Pattern variables connect these ideas: their scope includes only regions that can execute after the match is known to have succeeded. Control-flow reasoning becomes reliable when each branch, short-circuit edge, abrupt exit, loop back edge, break, and switch rule is traced explicitly.

1. Blocks define lexical regions for local names

A block is a sequence of declarations and statements enclosed by braces. The scope of a local declared by a statement in a block begins with its own initializer and continues through the rest of that block. Code outside the block cannot refer to the local by its simple name. An outer local remains in scope inside a nested block unless a language rule prevents or changes name resolution.

Java does not permit a nested local declaration to reuse an outer local’s name while the outer declaration remains in scope. A local may, however, shadow a field; `this.name` still selects the field. That distinction is a name-resolution rule, not a claim that the field or outer local stopped existing.

Predict before running: Which assignments update `total`? Can `increment` be named after the nested block closes?

```
int total = 1;
{
    int increment = 2;
    total += increment;
}

System.out.printf("scope: total=%d\n", total);
```

Output:

```
scope: total=3
```

The inner declaration creates a new local variable whose `int` value participates in Chapter 2’s compound-assignment operation. The assignment target is the outer `total` local, so its stored value becomes 3. `increment` is no longer in scope after the closing brace; an attempted read there fails compilation even though the value 2 existed during the block’s execution.

Scope also differs from lifetime. A reference copied from a local into a field or returned to a caller can keep an object reachable after the local’s scope ends. Conversely, a local name can remain in scope in source text that a particular run never reaches. Chapter 1’s object graph and this chapter’s control-flow graph answer different questions: the first tracks identities and aliases; the second tracks possible execution edges and name availability.

Use blocks to make temporary names and invariants local to the code that needs them. A smaller scope reduces the number of statements that can read or reassign a local, but braces alone do not make a referenced object immutable or prevent aliases outside the block.

2. Boolean operators remove or preserve execution paths

An `if`, loop condition, or conditional expression evaluates a `boolean` expression to choose a path. Chapter 2 established left-to-right expression evaluation. The conditional-and operator `&&` evaluates its right operand only when the left operand is true; conditional-or `||` evaluates its right operand only when the left operand is false. These operators therefore control side effects as well as values.

Predict before running: How many calls to `record` occur? Which operand is skipped, and why is the final result true?

```
int[] calls = {0};

boolean accepted =
    false && record(calls)
    || true && record(calls);

System.out.printf(
    "paths: accepted=%b calls=%d\n",
    accepted,
    calls[0]
);
```

Output:

```
paths: accepted=true calls=1
```

`&&` groups more tightly than `||`. The first `false && ...` cannot become true, so its right operand does not run. The left side of `||` is false, so evaluation continues to `true && record(calls)`. That second `&&` must evaluate `record`, which increments the array component and returns true. The final result is true, and only one mutation

occurs.

Short-circuit structure can prove safety. In `reference != null && reference.ready()`, the method invocation is reached only after the non-null comparison succeeds. Reversing the operands removes that protection because the object-requiring invocation becomes the first operation. The compiler recognizes some path facts for pattern variables and definite assignment, but it does not generally prove arbitrary application predicates such as a collection being nonempty or an integer being in range. Those remain explicit program invariants.

For human reasoning, treat each `&&` and `||` as a branch edge. If an operand has a side effect, record both the value edge and the state change. A compound condition can be specified yet difficult to audit; named booleans or guard clauses can expose the same path structure more clearly.

3. Local reads require definite assignment

Fields and array components receive default values during object or array initialization. Local variables do not receive a comparable default-value guarantee. Before a local is read, the compiler must establish that the local is **definitely assigned**: every possible execution path to that read includes an assignment to it. The analysis follows language-defined structural rules rather than running the program with likely inputs.

Predict before running: Why is `selected` accepted even though it has no initializer? Why can `holder.count` be read before an explicit field assignment?

```
boolean primary = false;
int selected;

if (primary) {
    selected = 10;
} else {
    selected = 20;
}

Holder holder = new Holder();
System.out.printf(
    "assignment: selected=%d field=%d\n",
    selected,
    holder.count
);
```

Output:

```
assignment: selected=20 field=0
```

Exactly one branch runs for this invocation, and both branches assign `selected`. After the complete `if-else`, every incoming path to the output statement has therefore assigned the local. The run-time value of `primary` chooses 20; the compiler's proof does not depend on that particular value. The newly created `Holder` has an `int` field whose default value is zero under the separate object-initialization rules acquired in Chapter 5.

Removing the `else` changes the proof. Even if tests always call the code with `primary == true`, the structural false path reaches the later read without an assignment. `examples/ch03/test/NotDefinitelyAssigned.java.txt` verifies the resulting compilation error. Initializing every local with a dummy value can silence the diagnostic, but it may replace a useful proof obligation with an invalid state. Prefer branch structure that assigns a meaningful value on each continuing path, or exit abruptly on paths that cannot supply one.

Definite assignment also supports blank `final` locals: the compiler must prove assignment before use and definite unassignment before the one permitted assignment. This analysis restricts writes to the variable; Chapter 1's boundary still applies when the final value is a reference to a mutable object.

4. Abrupt completion changes later reachability

A statement completes normally when control reaches the statement's end in the ordinary way. `return`, `throw`, `break`, and `continue` are forms of abrupt completion with different destinations or reasons. If every path through a statement completes abruptly, the next statement in the same block is not reachable through that statement.

Reachability is a technical compile-time property. Java generally analyzes statement structure rather than evaluating arbitrary expression values. A statement may be reachable even when a programmer knows that current data makes its path impossible. Conversely, a statement after an unconditional `return` or after a `while` (`true`) loop with no reachable `break` is structurally unreachable and causes a compilation error.

Predict before running: Which guards complete abruptly for each argument, and which final return remains reachable?

```
System.out.printf(
    "guards: negative=%s zero=%s positive=%s\n",
    sign(-2),
    sign(0),
    sign(3)
);
```

Output:

```
guards: negative=negative zero=zero positive=positive
```

The method is:

```
static String sign(int value) {
    if (value < 0) {
        return "negative";
    }
    if (value == 0) {
        return "zero";
    }
    return "positive";
}
```

For a negative value, the first `return` prevents the second condition and final return from executing. A zero value follows the first false edge, then returns from the second guard. Only a positive value reaches the final return. The final statement is nevertheless reachable because the preceding `if` statements can complete normally on their false paths.

Guard clauses often simplify definite-assignment and scope proofs: reject or return on a path that cannot establish a required fact, leaving subsequent code with fewer incoming paths. The conclusion is local. A `finally` clause can affect transfers, and Chapter 11 owns that exception interaction. Here the important move is to mark every statement that cannot complete normally and follow only the edges it actually permits.

5. Pattern-variable scope follows successful matches

A type pattern such as input `instanceof String text` both tests a value and declares a pattern variable. The variable is initialized by successful matching, so its scope includes only program regions that can execute when that match is known to have succeeded. This is **flow scoping**: scope depends on control-flow facts, not only on surrounding braces.

In input `instanceof String text && !text.isBlank()`, `text` is in scope in the right operand because reaching it proves the left operand was true. Replacing `&&` with `||` does not supply that proof: the right operand may run precisely when matching failed. The compiler therefore rejects a use of `text` there.

Predict before running: Why is `text` in scope after the first guard? What does each input return?

```
System.out.printf(
    "patterns: text=%s number=%s%n",
    normalize(" Java "),
    normalize(42)
);
```

Output:

```
patterns: text=JAVA number=other
```

The method is:

```
static String normalize(Object input) {
    if (!(input instanceof String text)) {
        return "other";
    }
    if (text.isBlank()) {
        return "blank";
    }
    return text.trim().toUpperCase();
}
```

When the pattern fails, the first branch returns and cannot complete normally. Any path that reaches the second `if` must therefore come from a successful match, so `text` is in scope through the rest of the method. It is also initialized by the match; pattern variables do not need a separate definite-assignment proof before those scoped reads.

Flow scoping is not an assertion that `text` exists for the entire method invocation. For a non-string input, no continuing execution path reaches a use of that name. `examples/ch03/test/PatternOutOfScope.java.txt` proves that placing a read after an ordinary positive `if` is invalid when the false branch can continue. The compiler is tracking possible paths, not moving one declaration to method scope.

6. Loop exits determine what is known afterward

Loops add back edges: after the body, control may return to the condition or update rather than proceed to the following statement. A `while` or basic `for` loop may execute zero times, so an assignment only in its body normally does not definitely assign a local after the loop. A `do` loop executes its body before its first condition test, but `continue`, `break`, and abrupt exits still affect the proof.

An infinite loop with reachable breaks can establish a post-loop fact. To read a local after that loop, every break that exits to the read must assign the local first. Paths that continue do not reach the read yet; paths that return or throw never reach it.

Predict before running: How many iterations occur? Why is `state` definitely assigned after the loop even though its declaration has no initializer?

```
int attempts = 0;
String state;

while (true) {
    attempts++;
    if (attempts < 3) {
        continue;
    }
    state = "ready";
    break;
}

System.out.printf(
    "loop: attempts=%d state=%s%n",
    attempts,
    state
);
```

Output:

```
loop: attempts=3 state=ready
```

The first two iterations take `continue`, which begins the next condition check and cannot reach the output statement. The third iteration assigns `state` and then takes the only break that exits the loop. The compiler can therefore prove that every path reaching the output through that break has assigned `state`.

This is not a proof that an arbitrary loop terminates. For this example, termination follows from the small integer trace, while definite assignment follows from the structure of reachable exits. In production code, keep progress, exit conditions, and post-loop state visible. A loop whose correctness depends on a hidden mutation in its condition is harder to audit even when it is legal.

7. Switch form determines fall-through and result paths

A `switch` selector chooses among labels. Traditional colon-labeled statement groups permit execution to continue through later labels until a break or another abrupt completion occurs. Arrow rules execute only the selected rule and do not fall through. Choose deliberately: fall-through can combine cases, but an omitted break can also create an unintended path.

A `switch` expression must handle every possible selector value and produce a result on every selected path, either through an expression or `yield`; a selected path may instead throw. That requirement makes the value-producing control flow explicit and often avoids a separate blank local with branch-by-branch assignments.

Predict before running: Which colon-labeled groups execute for `level == 2`? Which switch-expression rule then supplies `band`?

```

int level = 2;
int score = 0;

switch (level) {
    case 2:
        score += 2;
    case 1:
        score += 1;
        break;
    default:
        score = -1;
}

String band = switch (score) {
    case 3 -> "high";
    case 1, 2 -> "medium";
    default -> "other";
};

System.out.printf(
    "switch: score=%d band=%s%n",
    score,
    band
);

```

Output:

```
switch: score=3 band=high
```

Label 2 applies, adds two, and falls through to the statements after label 1. Those add one and break, so the default group does not execute. The switch expression then selects the `case 3` arrow rule and produces `"high"`. No switch-expression rule can complete normally without yielding its required value.

Pattern labels add flow-scoped variables to the selected rule; Chapter 7 combines that mechanism with sealed hierarchies. The boundary here is structural: identify every label that can enter a statement group, every path that falls through, and every abrupt exit. Do not infer exclusivity merely because the source contains separate `case` labels.

False models, repaired

- **“Scope is how long a value lives.”** Scope is a region of program text for name use. An object may remain reachable through another reference after a local name leaves scope.
- **“Locals get the same defaults as fields.”** Fields and array components receive initialization defaults. A local must be definitely assigned before each read.
- **“If tests always take the assigning branch, the local is initialized.”** The compiler requires an assignment on every structurally possible path to the read, independent of test coverage.
- **“Reachable means this input will execute the statement.”** Reachability is a compile-time structural property; run-time values choose one of the admitted paths.
- **“A pattern variable belongs to the surrounding block.”** Its scope includes only regions where control flow proves the pattern matched.
- **“A loop-body assignment initializes the local after the loop.”** A pre-test loop may run zero times; every exit path to the read must be analyzed.
- **“Every case is isolated.”** Colon-labeled switch groups allow fall-through. Arrow rules do not.
- **“`break` completes normally because execution continues afterward.”** The `break` statement completes abruptly; its target may then complete normally and permit a later statement.

Mastery questions

Commit to a path trace and compile result before checking any constructed case.

1. A local reference leaves scope while an alias remains in a field. What disappeared, what remained, and which chapter model explains each fact?
2. Predict the calls in `ready() || record() && verify()`. State the grouping and every condition under which each method runs.
3. Determine whether a local assigned only in the `then` branch of an `if` is definitely assigned afterward. Change the false path in the smallest way that makes a later read legal without inventing a dummy value.
4. Compare a new object's `int` field with an uninitialized local `int`. Which operation supplies one value, and which compiler rule rejects the other read?
5. Why can a statement be reachable even when a preceding condition is known by the programmer to be false for all valid inputs?
6. Trace the paths through two guard clauses where the first throws, the second returns, and the last statement assigns a result. Which paths can reach the assignment?
7. Explain why `value instanceof String text && text.length() > 2` can use `text`, while the corresponding `||` expression cannot.
8. Rewrite a positive pattern `if` so the pattern variable is in scope after the statement. Name the abrupt-completion fact the compiler uses.
9. A `while (condition)` assigns `result` on every iteration. Why might `result` still be unassigned afterward? Give two repairs with different contracts.
10. For an infinite loop with three `break` statements, formulate the definite-assignment obligation for a local read after the loop.
11. Predict the result of a colon-labeled switch whose matched group lacks a break and flows through two later labels. Then rewrite it with arrow rules preserving the intended result.
12. Explain how a switch expression's exhaustiveness and result requirement can replace a blank local plus later assignment.
13. Construct the smallest unreachable-statement compilation failure and a nearby reachable version. Identify the structural edge restored by the repair.
14. Design a classifier for `null`, nonblank strings, nonnegative integers, and all other objects. Mark the scope and assignment state of every local on each path.

The closed-book workspace, transfer cases, mismatch-report format, and retrieval schedule are in `exercises/ch03.md`.

Implementation lab — Flow-Safe Status Rendering

Implement `StudentStatusRenderer.render` in `examples/ch03/lab/`. Declare one blank final local `base`, assign it exactly once across an `if/else if/else` chain, and use pattern variables only where their matches are known to have succeeded. A nonblank `String` becomes its trimmed content; a nonnegative `Integer` becomes `count:<value>`; every other input, including null and blank strings, becomes `invalid`. After the branches join, uppercase the complete `base` only when requested.

Before coding, mark every path to the final transformation and prove that `base` is both definitely assigned and assigned at most once. Run the evaluator against the reference implementation, then against `FaultyStatusRenderer`, which treats a blank string as valid. Identify the missing predicate edge before discussing the later empty result.

Presentation prompt

Give a 10–15 minute explanation from sparse notes. Reconstruct the relationship among statement paths, scope, reachability, definite assignment, and run-time choice. Include one short-circuit side-effect prediction, a successful and failed local-assignment proof, a guard-clause reachability trace, a pattern-variable scope derivation, a loop with `continue` and `break`, a colon-switch fall-through case, and one fact the compiler does not establish about application data or termination. Connect the explanation to Chapters 1 and 2. The recording checklist is in `presentations/prompts/ch03.md`.

Ready-when test

Proceed when, without the chapter open, you can determine whether an unfamiliar statement is reachable, whether a local or pattern variable is in scope, whether every path to a read definitely assigns it, and which path current values execute; distinguish field defaults from local assignment; trace short-circuit and abrupt-completion edges; prove post-loop assignment from every reachable exit without confusing that proof with termination; distinguish switch fall-through from arrow rules; implement and pass the status-rendering lab; diagnose the faulty renderer at its missing blank-string predicate; and state what compile-time flow analysis establishes and what remains a run-time or domain obligation.

Personal appendix index

No personal appendices have been generated. This is expected at the gold-chapter milestone.

Glossary

This glossary records the canonical sense of load-bearing terms. Chapters still define each term at first consequential use.

- **alias** — one of two or more access paths whose current reference values refer to the same object.
- **argument** — an expression supplied at an invocation; its evaluated value initializes a new parameter variable.
- **assignment** — an operation that stores a compatible value in a variable, field, or array component identified by the left-hand operand.
- **abrupt completion** — termination of expression or statement evaluation for a specified reason, such as throwing an exception, before normal completion produces its expected result.
- **boxing conversion** — conversion of a primitive value to a reference value of the corresponding wrapper type, with only limited identity guarantees for specified constant values.
- **block** — a brace-delimited sequence of declarations and statements that supplies a scope boundary and one unit of statement completion.
- **compile-time type** — the type assigned by the language rules to a variable or expression during compilation; it constrains accepted operations and possible values.
- **control-flow path** — one possible sequence of expression and statement evaluations admitted by the program's branch, loop, and abrupt-completion structure.
- **conversion context** — a syntactic and typing context, such as assignment, invocation, casting, string, or numeric context, that determines which conversions may apply to an expression.
- **declared type** — the type written or inferred for a declaration, considered as a compile-time type.
- **definite assignment** — the compile-time property that every possible execution path to a local-variable read or blank-final use includes a required assignment.
- **definite unassignment** — the compile-time property that no possible execution path to an assignment of a blank final variable includes an earlier assignment to that variable.
- **flow scoping** — determination of pattern-variable scope from program regions that can execute only after the pattern is known to have matched.
- **identity** — the property by which an object remains that particular object despite state changes and differs from another object with equal-looking state.
- **expression** — a language construct whose evaluation denotes a variable, produces a value, or produces no result for a `void` invocation; evaluation may also cause side effects or complete abruptly.
- **immutability** — a contract that specified state cannot change after a defined point; the contract must name its depth, enforcement mechanism, and observable boundary.
- **mutation** — a state change to an existing object, including a write to one of its instance fields or array components.
- **narrowing primitive conversion** — conversion to a primitive numeric type with a smaller or otherwise different representable set, using the language's defined bit retention, rounding, clamping, or NaN rule rather than application validation.
- **normal completion** — completion of every required evaluation step without an exception or another abrupt reason.
- **numeric promotion** — the conversion of numeric operands to the primitive type in which a numeric operator or numeric choice is evaluated.
- **object** — a class instance or an array at run time.
- **overflow** — a mathematical integer result outside the selected primitive integer type's range; Java integer

operators retain the result modulo the operation width and do not signal the condition by themselves.

- **parameter** — a call-local variable created for an invocation and initialized with the corresponding argument value.
- **primitive value** — a non-reference value of one of Java's primitive types.
- **reference type** — a class, interface, type-variable, or array type whose values are references or the null reference.
- **reference value** — a value that refers to an object, or the null reference, which refers to no object.
- **reachability** — the compile-time structural property that a statement has a possible execution path under the language's reachability rules; it does not say that current input values will take that path.
- **reassignment** — an assignment that changes the value stored in an already existing variable, field, or array component.
- **run-time class** — the class of the object referred to by a non-null reference value during execution.
- **shallow copy** — a new outer object whose component values are copied without recursively copying every referenced object.
- **short-circuit evaluation** — conditional evaluation in which `&&` skips its right operand after a false left value and `||` skips its right operand after a true left value.
- **scope** — the region of program text within which a declaration can be referred to by a simple name, subject to shadowing and flow-scope rules.
- **state** — the values stored in an object's instance fields or array components at a given point in execution.
- **statement** — a language construct executed for its effect and control transfer rather than for a value; it may complete normally or abruptly.
- **unboxing conversion** — conversion of a wrapper reference to its corresponding primitive value; conversion of a null wrapper reference throws `NullPointerException`.
- **widening primitive conversion** — conversion to a primitive numeric type with a broader representable range or format; some widening conversions to floating point can still lose precision.
- **variable** — a typed storage location. Local variables, parameters, fields, and array components are distinct kinds with different creation and initialization rules.